VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE
Faculty of Information Technology

REPORT
LAB 1: HDFS

COURSE: INTRODUCTION TO BIG DATA

LECTURER:

- Nguyen Ngoc Thao
- Huynh Lam Hai Dang
- Tran Huy Ban

CLASS: 23KHDL2

STUDENT INFORMATION:

- Bui Nam Viet - Student ID: 23127516

TP.HCM, July 10, 2026.

Contents

1 HADOOP PSEUDO-DISTRIBUTED CLUSTER SETUP (SINGLE NODE)

1.1 Java Environment Setup (OpenJDK 17)

Hadoop runs on the Java platform, so a compatible Java environment must be installed before the cluster is configured. This lab uses **OpenJDK 17** on WSL 2 (Ubuntu).

The setup was completed as follows:

1.1.1 Step 1: Update package lists and system packages

The Ubuntu package index was updated, and installed packages were upgraded before installing Hadoop dependencies:

```
sudo apt update
sudo apt upgrade -y
```

Explanation: `apt update` refreshes package metadata, while `apt upgrade -y` installs available updates without additional prompts.

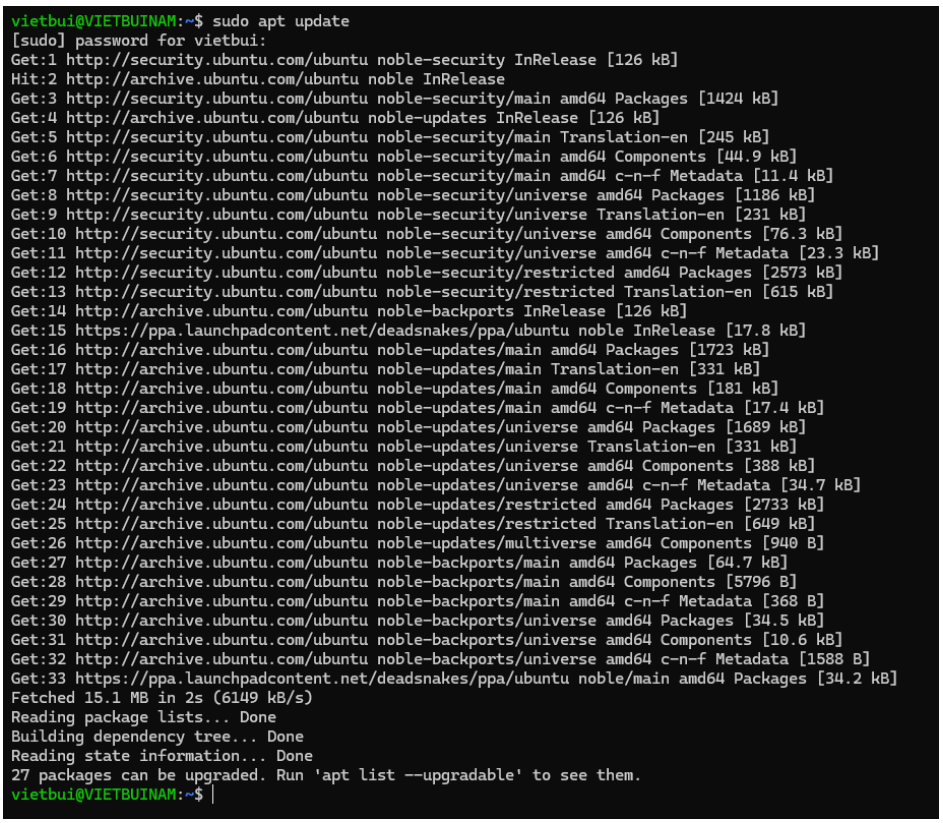

Figure 1: System updating on WSL Ubuntu.

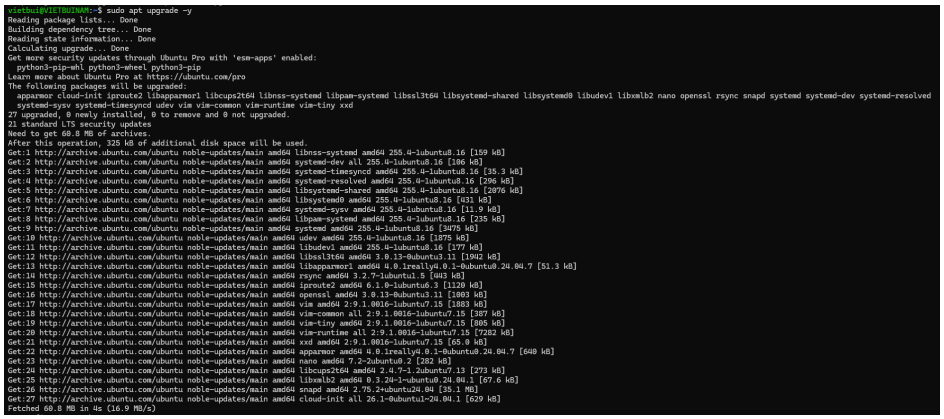

Figure 2: System upgrading on WSL Ubuntu.

1.1.2 Step 2: Install Java Development Kit (OpenJDK 17 JDK)

After the system update, the full OpenJDK 17 JDK package was installed:

```
sudo apt install openjdk-17-jdk -y
```

Explanation: The `-y` flag accepts the installation and required dependencies automatically.

Figure 3: Java installation on WSL Ubuntu.

1.1.3 Step 3: Verify installation success (Verification)

The Java installation was verified by checking the runtime version:

```
java -version
```

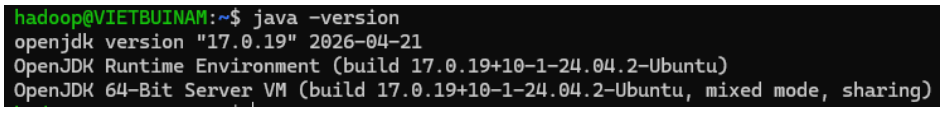

Figure 4: Java version verification on WSL Ubuntu.

- **Result:** The system displayed `openjdk version "17.0.19"` with the 64-Bit Server VM, confirming that Java was installed correctly and ready for Hadoop configuration.

1.2 Creating a Dedicated User and Configuring Passwordless SSH

Hadoop uses SSH to manage cluster processes, even in pseudo-distributed single-node mode. A dedicated `hadoop` user was created, and passwordless SSH was configured to support automated service startup.

1.2.1 Step 1: Create user and grant administrative privileges

A separate Linux user was created for the Hadoop environment and granted administrative privileges:

```
sudo adduser hadoop
sudo usermod -aG sudo hadoop
```

Explanation: The first command creates the `/home/hadoop` account environment. The second command adds the user to the `sudo` group, allowing administrative commands when required. The session was then switched to the new user using `sudo su - hadoop`.

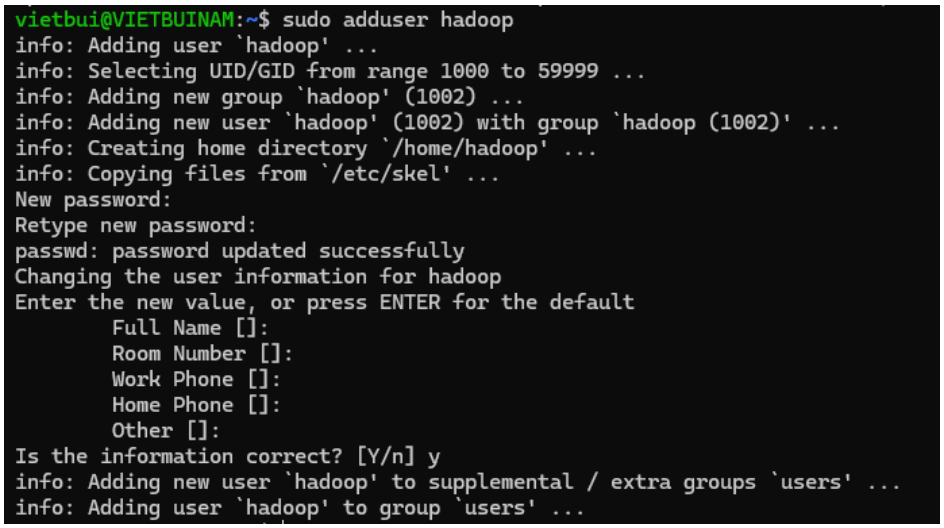

Figure 5: Creating a new user in WSL Ubuntu.

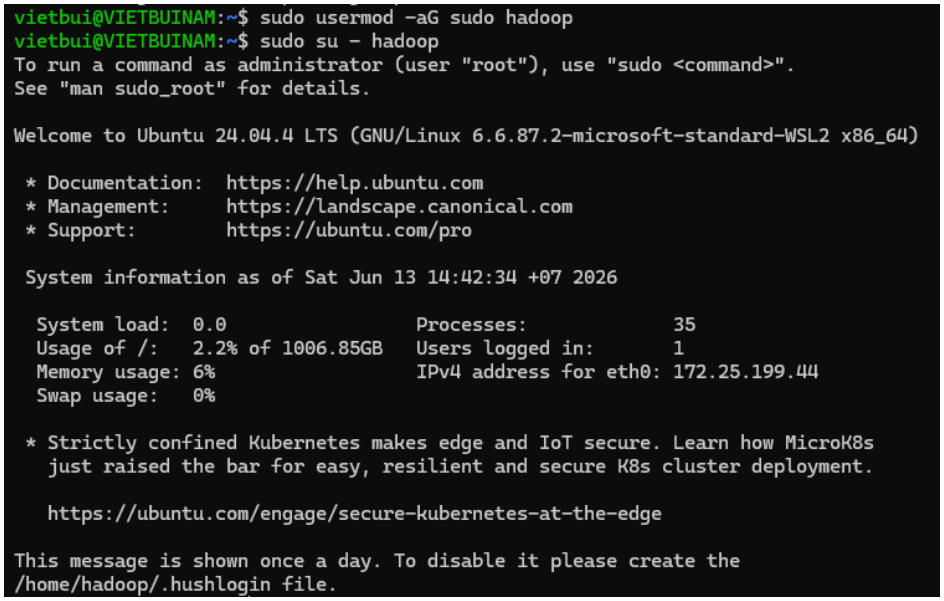

Figure 6: Granting sudo privileges and switching to hadoop user workspace.

1.2.2 Step 2: Install OpenSSH service

The SSH service package was installed to enable secure local communication:

```
sudo apt install ssh
```

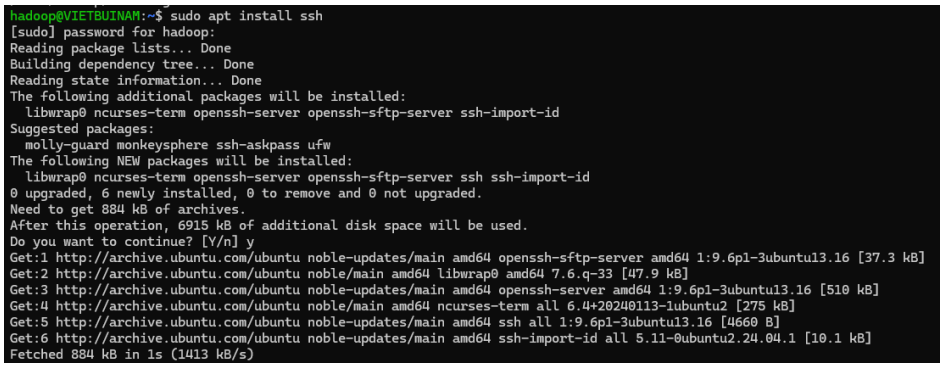

Figure 7: Installing OpenSSH service on WSL Ubuntu.

1.2.3 Step 3: Generate RSA key pair (SSH Key Generation)

An RSA key pair was generated for passwordless authentication:

```
ssh-keygen -t rsa
```

Note: The default key location was used, and the passphrase was left empty so Hadoop services could connect without manual input.

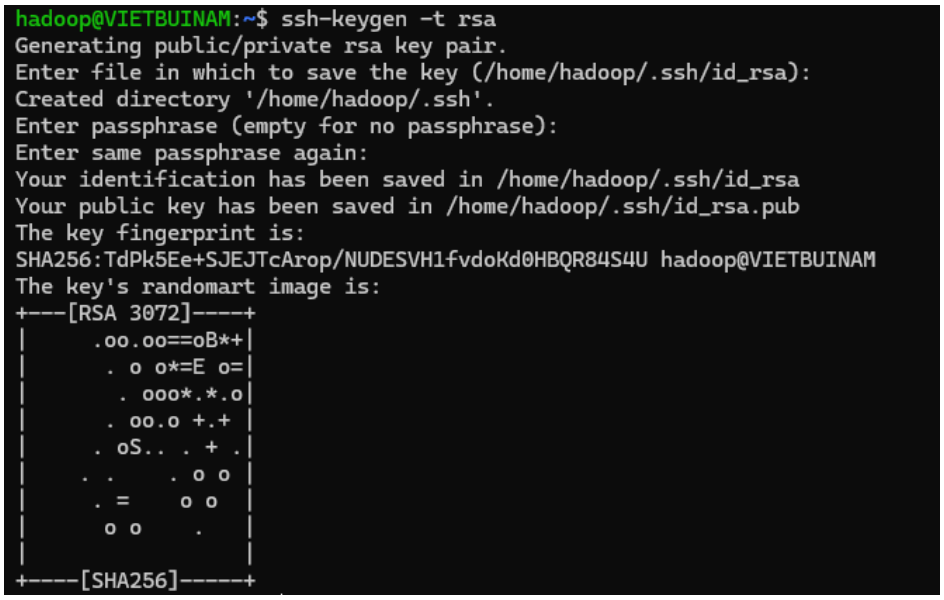

Figure 8: Successful RSA key generation with randomart visualization.

1.2.4 Step 4: Authorize key and configure file system permissions

The generated public key was added to `authorized_keys`, file permissions were configured, and the SSH service was started:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
sudo chmod 640 ~/.ssh/authorized_keys
sudo service ssh start
```

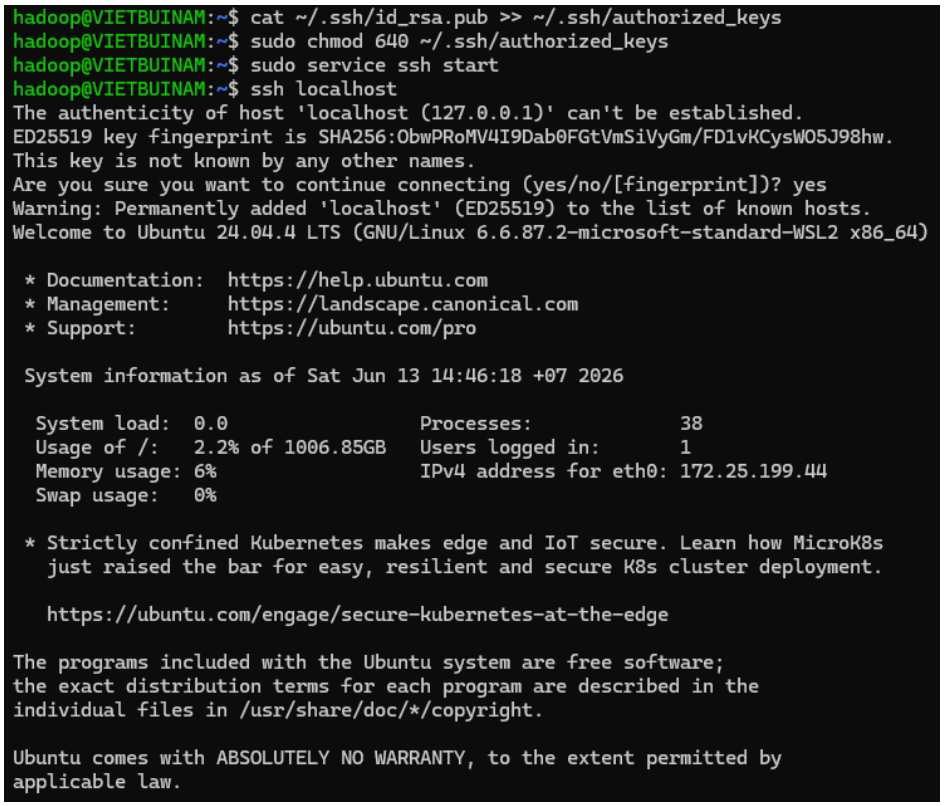

Figure 9: Adding the public key to `authorized_keys` and starting the SSH service.

Issue encountered: Incorrect permissions on SSH authorized keys

During passwordless SSH configuration, the command

```
sudo chmod 640 ~/.ssh/authorized_keys
```

caused the SSH server to reject public-key authentication. Although the mode appeared restrictive, using `sudo` with `640` produced an ownership and permission state that SSH did not accept for `authorized_keys`.

SSH requires strict ownership and permission settings for both the `~/.ssh` directory and the `authorized_keys` file. If these settings are too permissive or assigned to the wrong owner, the SSH daemon disables public-key authentication for security.

The permissions were corrected under the `hadoop` account without using `sudo`:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

The mode `700` restricts access to the `.ssh` directory to the owner, and `600` restricts `authorized_keys` to owner read/write access. After this correction, SSH accepted public-key authentication.

1.2.5 Step 5: Verify successful Passwordless SSH connection

The SSH configuration was tested by connecting to localhost:

```
ssh localhost
```

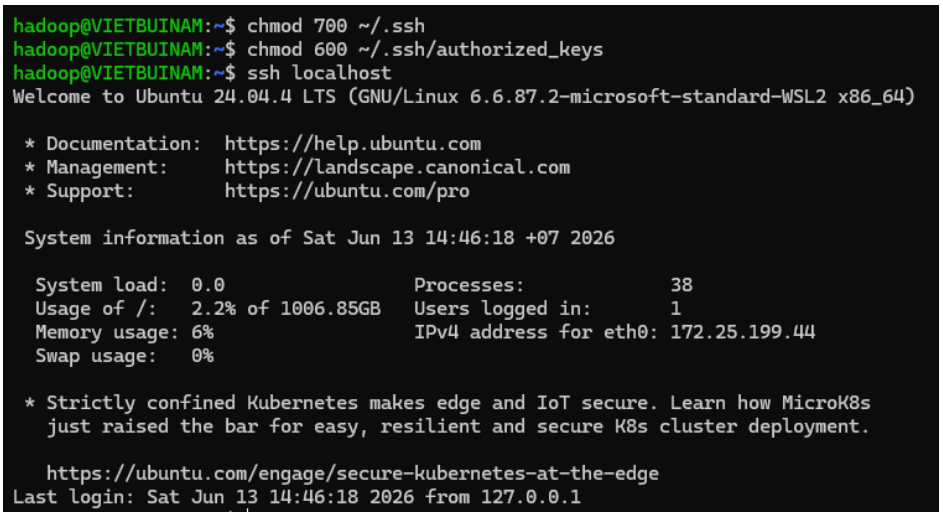

Figure 10: Typing `yes` to confirm the host fingerprint and successful passwordless login.

- **Result:** On the first connection, the system displayed a host fingerprint prompt. After entering `yes`, the host was stored in `known_hosts`, and the SSH client logged into `hadoop@VIETBUINAM` without requesting a password.

1.3 Downloading, Installing, and Configuring Environment Variables for Apache Hadoop

After Java and SSH were configured, the Apache Hadoop distribution was downloaded and environment variables were set so Hadoop commands could run from any working directory.

1.3.1 Step 1: Download Apache Hadoop package

The Hadoop binary distribution was downloaded from the official Apache repository:

```
wget https://downloads.apache.org/hadoop/common/stable/hadoop-3.5.0.tar.gz
```

Explanation: The file `hadoop-3.5.0.tar.gz` contains the Hadoop components required for HDFS and YARN in this lab.

Figure 11: Downloading Apache Hadoop package.

1.3.2 Step 2: Extract and organize directory structure

The downloaded archive was extracted and renamed for simpler path configuration:

```
tar -xzf hadoop-3.5.0.tar.gz
sudo mv hadoop-3.5.0 hadoop
```

Explanation: `tar -xzf` extracts the archive, and `mv` renames the extracted directory to `hadoop` in the user's home directory.

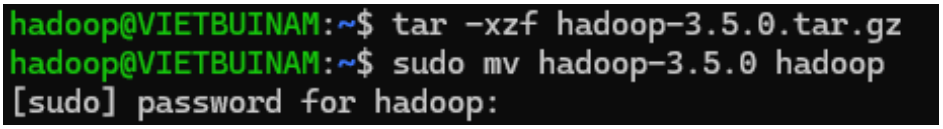

Figure 12: Extracting and renaming Hadoop directory.

1.3.3 Step 3: Assign ownership to target directory

Ownership of the Hadoop directory was assigned to the dedicated `hadoop` user:

```
sudo chown -R hadoop:hadoop ~/hadoop
```

Explanation: The recursive `-R` option applies ownership to all files and subdirectories, preventing permission errors during service startup.

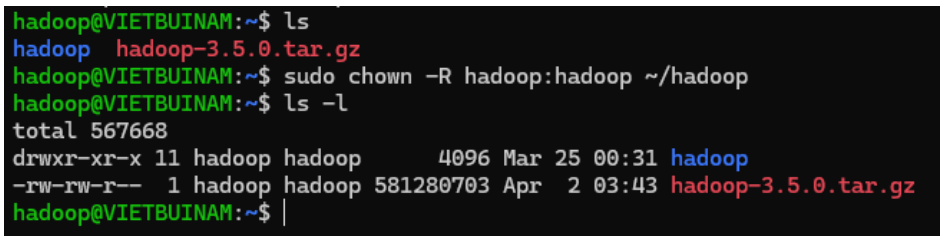

Figure 13: Verifying ownership using ls -l showing hadoop:hadoop.

1.3.4 Step 4: Configure environment variables via `.bashrc`

The `.bashrc` file was edited to define Hadoop and Java environment variables:

```
nano ~/.bashrc
```

The following configuration was appended to the file:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```

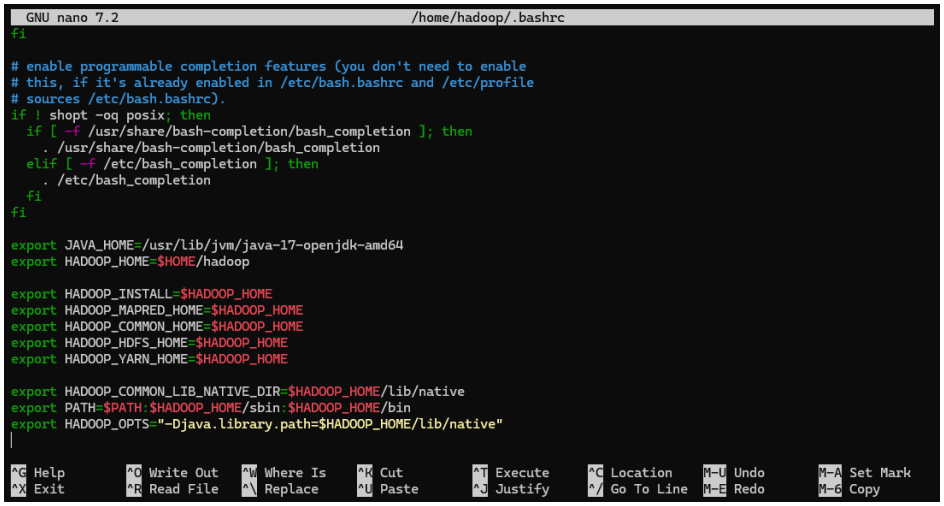

Figure 14: Editing .bashrc with Hadoop environment variables in nano.

1.3.5 Step 5: Activate environment variables

After saving the file in `nano`, the current shell session was reloaded:

```
source ~/.bashrc
```

- **Result:** The OpenJDK 17 and Apache Hadoop 3.5.0 paths were loaded into the shell environment. Commands such as `hdfs`, `hadoop`, and `start-dfs.sh` can now run from the `hadoop` user session.

1.4 Configuring Java Path in `hadoop-env.sh` and System Verification

Hadoop must reference the Java installation in its internal environment script so HDFS, YARN, and MapReduce services can start consistently.

1.4.1 Step 1: Set Java path in `hadoop-env.sh`

The `hadoop-env.sh` file in the Hadoop configuration directory was opened:

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

The `JAVA_HOME` setting was added for the installed OpenJDK 17 path:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

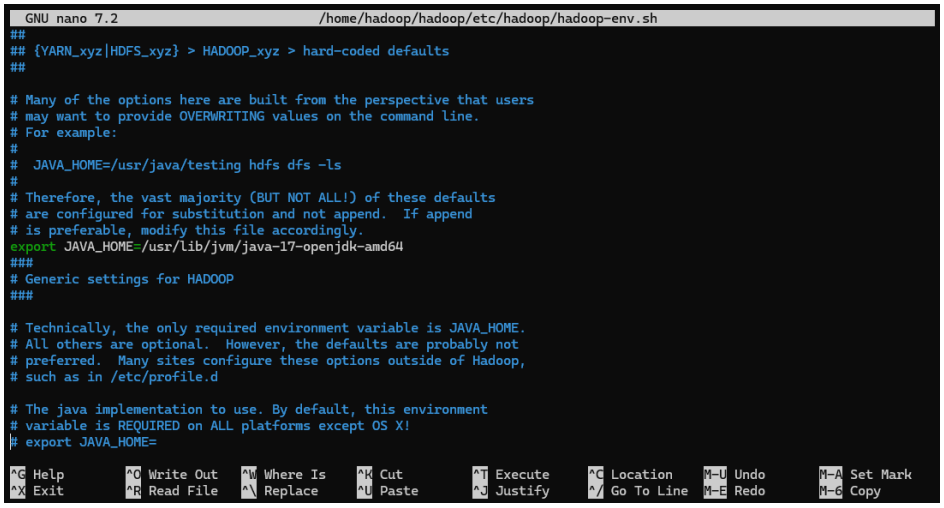

Figure 15: Editing hadoop-env.sh with Java path in nano.

1.4.2 Step 2: System version verification

The Hadoop version command was executed to verify the framework installation and environment configuration:

```
hadoop version
```

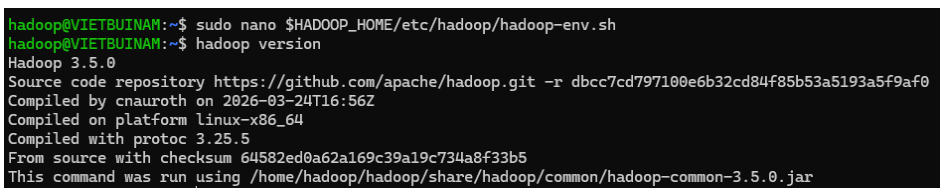

Figure 16: Output of hadoop version showing Apache Hadoop 3.5.0 core information.

- **Result:** The command returned Hadoop 3.5.0 information and the `hadoop-common-3.5.0.jar` path, confirming that the base Hadoop environment was configured correctly.

1.5 Configuring Core XML Files and Starting the Single-Node Hadoop Cluster

Pseudo-distributed mode requires the core Hadoop XML files to define HDFS access, local storage paths, MapReduce execution, and YARN services.

1.5.1 Step 1: Configure `core-site.xml`

`core-site.xml` was configured to define the default HDFS URI and communication port:

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
```

Inserted property: The default URI was set to `hdfs://localhost:9000`.

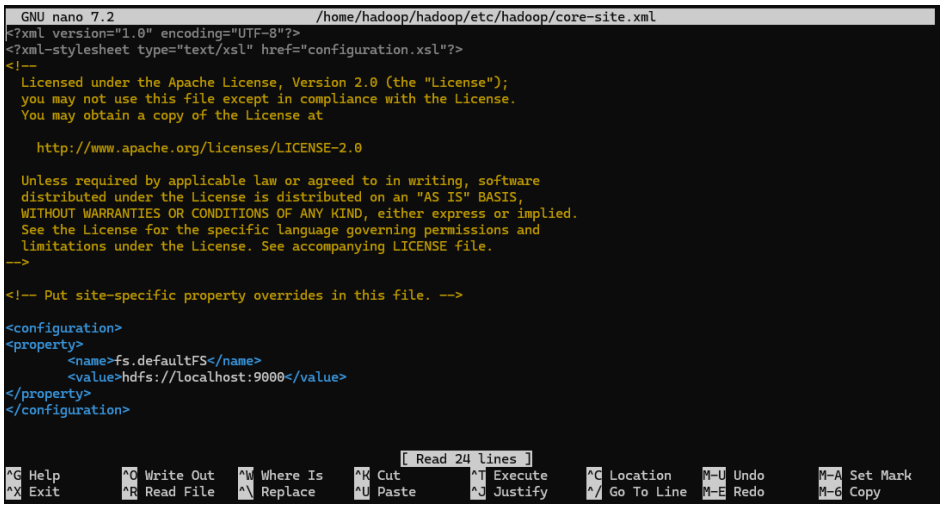

Figure 17: Configuring default filesystem property in core-site.xml.

1.5.2 Step 2: Configure `hdfs-site.xml`

Local directories were created for NameNode metadata and DataNode block storage, then added to `hdfs-site.xml`:

```
mkdir -p /home/hadoop/hdfs/{namenode,datanode}
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

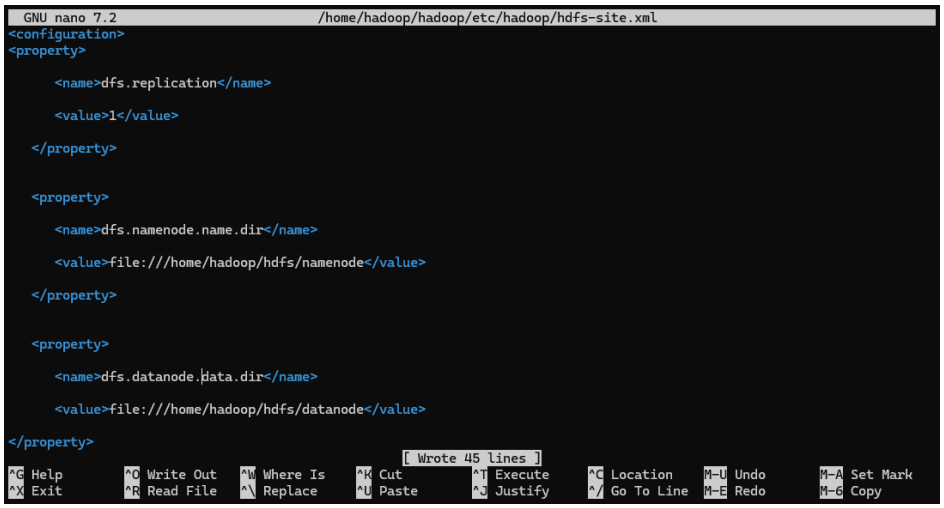

Figure 18: Configuring HDFS storage directories and replication factor.

Inserted properties: The replication factor `dfs.replication` was set to 1 for the single-

node setup, and the NameNode and DataNode storage paths were set to the created directories.

1.5.3 Step 3: Configure `mapred-site.xml` and `yarn-site.xml`

MapReduce was configured to use YARN for job scheduling and resource management:

```
nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

Inserted properties: In `mapred-site.xml`, the execution framework was set to `yarn`. In `yarn-site.xml`, `mapreduce_shuffle` was enabled to support the MapReduce Shuffle & Sort phase.

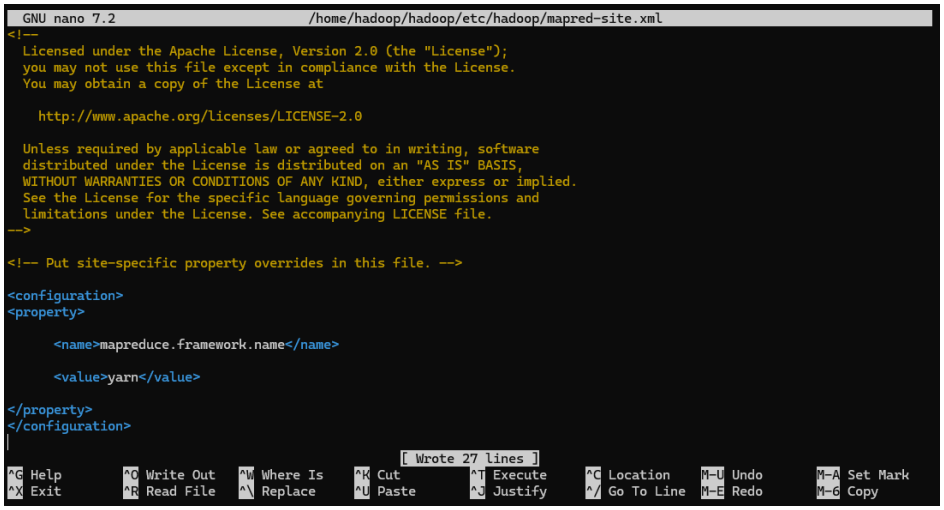

Figure 19: Configuring YARN framework in mapred-site.xml.

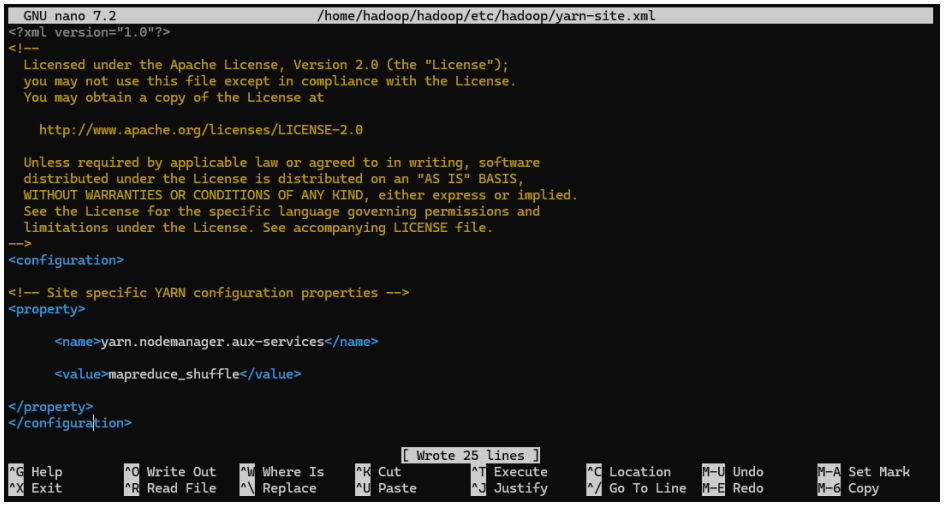

Figure 20: Enabling NodeManager shuffle service in yarn-site.xml.

1.5.4 Step 4: Format the NameNode (HDFS Format)

Before the first cluster startup, the NameNode was formatted to initialize HDFS metadata:

```
hdfs namenode -format
```

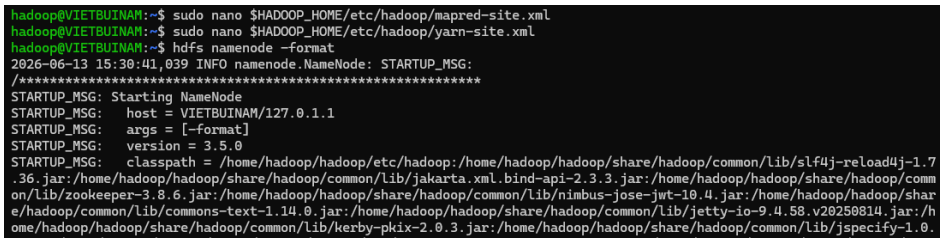

Figure 21: Formatting the NameNode metadata storage.

1.5.5 Step 5: Start all services and verify system status

The startup scripts were executed, and Java processes were checked to verify the cluster status:

```
start-dfs.sh
start-yarn.sh
jps
```

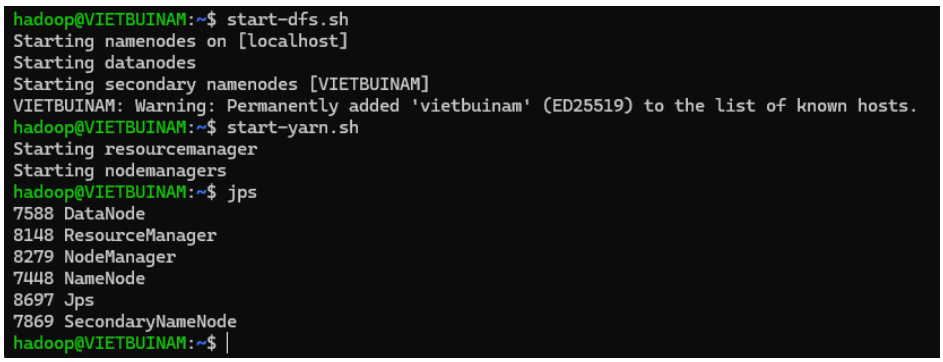

Figure 22: Successfully starting HDFS, YARN and jps showing all core processes.

- **Result:** The `jps` output showed **NameNode**, **SecondaryNameNode**, **DataNode**, **ResourceManager**, and **NodeManager** running under separate PIDs. This confirms that the pseudo-distributed Hadoop cluster was started successfully.

1.6 Cluster Status Verification via Web UI

Hadoop Web UI pages were used to confirm the HDFS and YARN status after startup.

1.6.1 Step 1: Explore NameNode Web UI

The NameNode Web UI was opened at `http://localhost:9870` to inspect HDFS status.

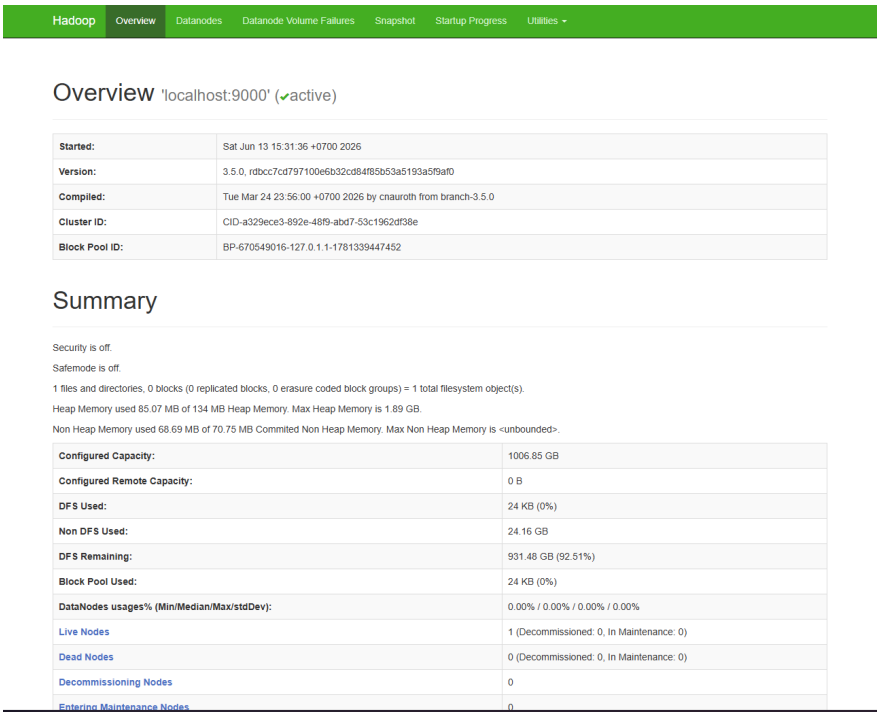

Figure 23: NameNode Web UI showing cluster status.

- **System Metrics Analysis:**
  - **State:** The interface showed the NameNode state as `active` and reported Hadoop version `3.5.0`, matching the installed environment.
  - **Summary:** The status `Safemode is off` indicates that HDFS is ready for read/write operations. The `Live Nodes` value was **1**, confirming that the single DataNode was connected, and `DFS Remaining` was 92.51%.

1.6.2 Step 2: Explore YARN ResourceManager Web UI

The ResourceManager Web UI was opened at `http://localhost:8088` to inspect YARN status.

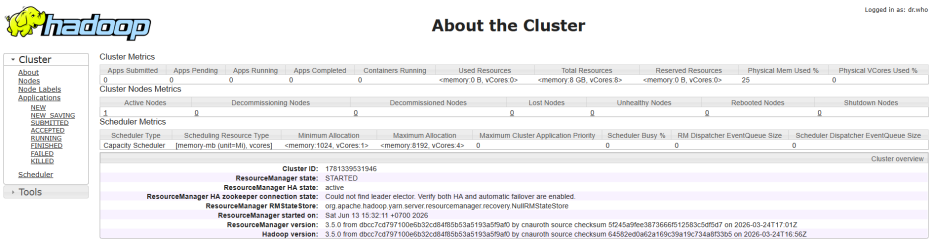

Figure 24: YARN ResourceManager Web UI showing cluster metrics and scheduler information.

- **System Metrics Analysis:**
  - **Cluster Metrics:** `Apps Running` and `Apps Completed` were both **0** because no processing jobs had been submitted. The `Active Nodes` value was **1**, indicating that the NodeManager was available for containers.
  - **Total Resources:** The interface reported `8 GB` virtual memory and `8 vCores`. The `ResourceManager HA state` was `active`, confirming that the scheduler was initialized.
- **Conclusion:** The Web UI metrics matched the local process verification, confirming

that the cluster was ready for HDFS operations through the CLI.

2 Pseudo-Distributed Mode

2.1 Step 1: Creating the System Directory and Dedicated User

- **Create the root directory:** The parent directory `/hcmus` was created on HDFS as the common storage workspace:

```
hdfs dfs -mkdir /hcmus
```

- **Create a dedicated system user:** A Linux user named `khtn_23127516` was created for ownership assignment:

```
sudo adduser khtn_23127516
```

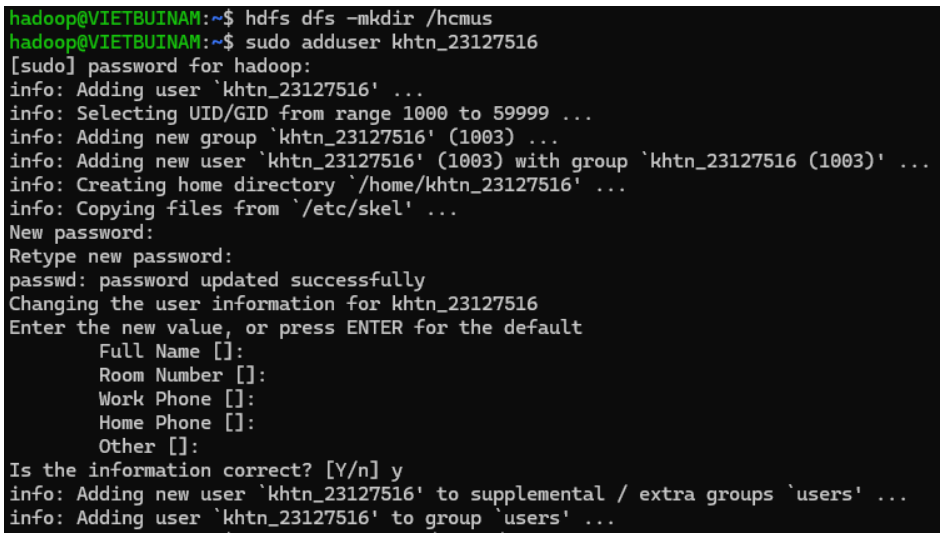

Figure 25: Creating the root directory `/hcmus` and configuring the user `khtn_23127516`.

2.2 Step 2: Managing Subdirectories, Uploading Data, and Configuring Permissions

- **Create the student directory:** The subdirectory `/hcmus/23127516` was created under the HDFS root directory hierarchy.
- **Upload data to HDFS:** A local text file named `sample.txt` was created and uploaded to HDFS:

```
echo "HDFS␣Test␣File" > sample.txt
hdfs dfs -put sample.txt /hcmus/23127516/
```

- **Configure permissions and transfer ownership:** Permission mode `744` was assigned, and ownership of the student directory was transferred to `khtn_23127516`:

```
hdfs dfs -chmod 744 /hcmus/23127516
hdfs dfs -chown khtn_23127516 /hcmus/23127516
```

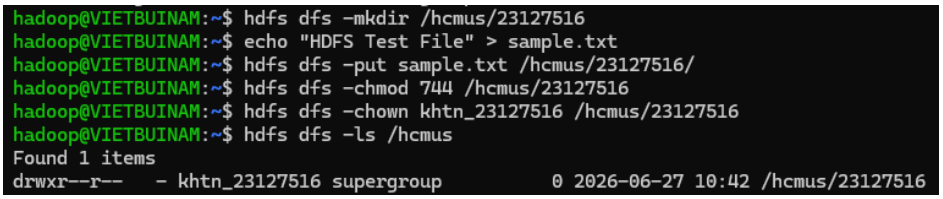

Figure 26: Executing data operations, applying `chmod 744`, and verifying the directory structure using the `ls` command.

- **Verification:** The command `hdfs dfs -ls /hcmus` confirmed the expected state. The directory showed permission attribute `drwxr--r--`, corresponding to mode `744`, and ownership was assigned to `khtn_23127516`.

2.3 Step 3: Executing the JAR Program for Automated System Verification

The verification JAR was executed from `/home/hadoop/run` to validate the HDFS configuration automatically.

- **Execution command:**

```
mkdir -p ~/run && cd ~/run
java -jar ~/hadoop-test.jar 9000 /hcmus/23127516
```

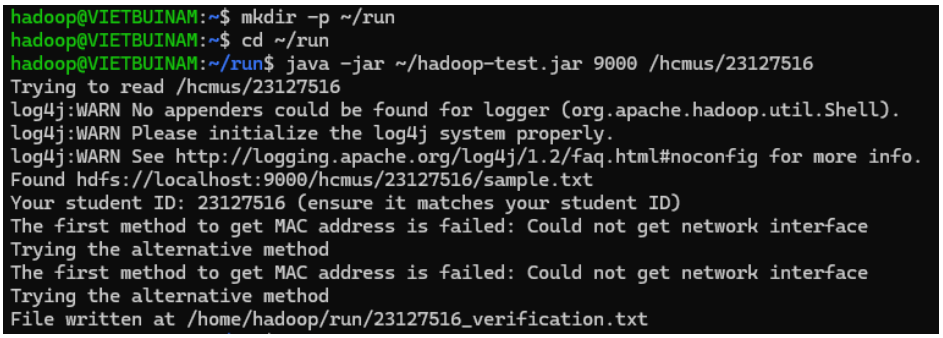

Figure 27: Successful execution of `hadoop-test.jar` generating the verification code.

3 Word Count

3.1 Step 1: Creating the HDFS Directory Structure and Uploading the Input Dataset

The HDFS input directory was created, and `words.txt` was uploaded for the Word Count job.

Commands executed:

```
hdfs dfs -mkdir -p /hcmus/23127516/warmup/input
hdfs dfs -put -f words.txt /hcmus/23127516/warmup/input/
hdfs dfs -ls /hcmus/23127516/warmup/input
```

```
hadoop@VIETBUINAM:~$ hdfs dfs -mkdir -p /hcmus/23127516/warmup/input
hadoop@VIETBUINAM:~$ hdfs dfs -ls
Found 1 items
drwxr-xr-x   - hadoop supergroup          0 2026-07-05 18:19 input
hadoop@VIETBUINAM:~$ hdfs dfs -mkdir -p /hcmus/23127516/warmup/input
hadoop@VIETBUINAM:~$ hdfs dfs -mkdir -p /hcmus/23127516/warmup/output
hadoop@VIETBUINAM:~$ hdfs dfs -put -f words.txt /hcmus/23127516/warmup/input/
hadoop@VIETBUINAM:~$ hdfs dfs -ls /hcmus/23127516/warmup/input
Found 1 items
-rw-r--r--   1 hadoop supergroup    4862985 2026-07-06 22:15 /hcmus/23127516/warmup/input/words.txt
```

Figure 28: Creating the HDFS directory structure and uploading the input dataset `words.txt`.

3.2 Step 2: Compiling the Scala Application

`WordCount.scala` was compiled with the Hadoop classpath and packaged into `WordCount.jar`.

Commands executed:

```
scalac -classpath "$(hadoop␣classpath)" WordCount.scala
jar cvf WordCount.jar WordCount*.class
```

```
hadoop@VIETBUINAM:~$ nano WordCount.scala
hadoop@VIETBUINAM:~$ scalac -classpath "$(hadoop classpath)" WordCount.scala
hadoop@VIETBUINAM:~$ jar cvf WordCount.jar WordCount*.class
added manifest
adding: WordCount$.class(in = 2726) (out= 1384)(deflated 49%)
adding: WordCount$IntSumReducer.class(in = 2189) (out= 907)(deflated 58%)
adding: WordCount$TokenizerMapper$$anonfun$map$1.class(in = 2541) (out= 1255)(deflated 50%)
adding: WordCount$TokenizerMapper.class(in = 2553) (out= 991)(deflated 61%)
adding: WordCount.class(in = 1621) (out= 1257)(deflated 22%)
```

Figure 29: Successful compilation and packaging of the Scala application into a JAR file.

3.3 Step 3: Submitting the Job to YARN and Resolving the `ClassNotFoundException`

The first MapReduce submission failed inside the YARN containers with the following exception:

```
Error: java.lang.ClassNotFoundException: scala.Function1
```

```
hadoop@VIETBUINAM:~$ hadoop jar WordCount.jar WordCount /hcmus/23127516/warmup/input/words.txt /hcmus/
23127516/warmup/output
2026-07-06 22:36:34,401 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager
at /0.0.0.0:8032
2026-07-06 22:36:35,753 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not per
formed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-06 22:36:35,770 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/ha
doop-yarn/staging/hadoop/.staging/job_1783350631818_0004
2026-07-06 22:36:35,940 INFO input.FileInputFormat: Total input files to process : 1
2026-07-06 22:36:35,994 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-06 22:36:36,110 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783350631818_0004
2026-07-06 22:36:36,111 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-06 22:36:36,245 INFO conf.Configuration: resource-types.xml not found
2026-07-06 22:36:36,246 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-06 22:36:36,303 INFO impl.YarnClientImpl: Submitted application application_1783350631818_0004
2026-07-06 22:36:36,328 INFO mapreduce.Job: The url to track the job: http://VIETBUINAM.localdomain:80
88/proxy/application_1783350631818_0004/
2026-07-06 22:36:36,329 INFO mapreduce.Job: Running job: job_1783350631818_0004
2026-07-06 22:36:40,390 INFO mapreduce.Job: Job job_1783350631818_0004 running in uber mode : false
2026-07-06 22:36:40,391 INFO mapreduce.Job:  map 0% reduce 0%
2026-07-06 22:36:43,426 INFO mapreduce.Job: Task Id : attempt_1783350631818_0004_m_000000_0, Status :
FAILED
Error: java.lang.ClassNotFoundException: scala.Function1
        at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
        at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:525)
        at java.base/java.lang.Class.forName0(Native Method)
        at java.base/java.lang.Class.forName(Class.java:469)
        at org.apache.hadoop.conf.Configuration.getClassByNameOrNull(Configuration.java:2661)
        at org.apache.hadoop.conf.Configuration.getClassByName(Configuration.java:2626)
        at org.apache.hadoop.conf.Configuration.getClass(Configuration.java:2722)
        at org.apache.hadoop.mapreduce.task.JobContextImpl.getMapperClass(JobContextImpl.java:189)
        at org.apache.hadoop.mapred.MapTask.runNewMapper(MapTask.java:761)
        at org.apache.hadoop.mapred.MapTask.run(MapTask.java:349)
        at org.apache.hadoop.mapred.YarnChild$2.run(YarnChild.java:178)
        at java.base/java.security.AccessController.doPrivileged(AccessController.java:712)
        at java.base/javax.security.auth.Subject.doAs(Subject.java:439)
        at org.apache.hadoop.security.authentication.util.SubjectUtil.doAs(SubjectUtil.java:328)
        at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1958)
        at org.apache.hadoop.mapred.YarnChild.main(YarnChild.java:172)
```

Figure 30: System log reporting the missing Scala runtime library.

Analysis

The `hadoop jar` command loaded the Hadoop libraries, but the YARN container JVMs did not include the Scala runtime. As a result, the containers could not find `scala.Function1`, which is provided by `scala-library.jar`, and the Mapper tasks failed.

Solution

The Scala runtime path `/usr/share/scala-2.11/lib/*` was added to `yarn.application.classpa` in `yarn-site.xml`. The incomplete output directory was then removed, and YARN was restarted to apply the configuration.

Commands executed:

```
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

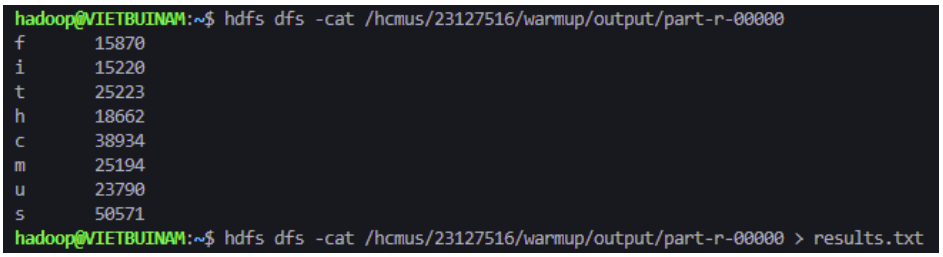

Figure 31: Adding the Scala runtime library path to the YARN classpath in `yarn-site.xml`.

```
hdfs dfs -rm -r -f /hcmus/23127516/warmup/output
stop-yarn.sh && start-yarn.sh
```

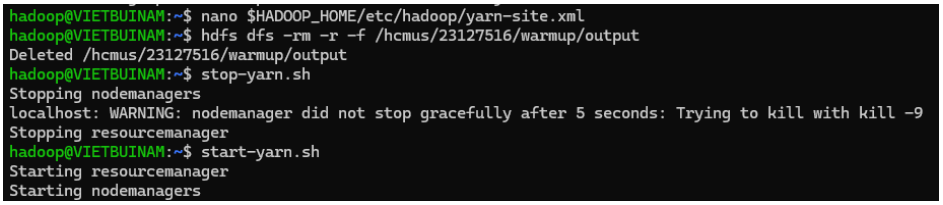

Figure 32: Removing the incomplete output directory and restarting the YARN cluster.

3.4 Step 4: Successful Job Execution

After the YARN classpath was updated, the MapReduce application was submitted again.

Command executed:

```
hadoop jar WordCount.jar WordCount \
/hcmus/23127516/warmup/input/words.txt \
/hcmus/23127516/warmup/output
```

```
hadoop@VIETBUINAM:~$ hadoop jar WordCount.jar WordCount /hcmus/23127516/warmup/input/words.txt /hcmus/
23127516/warmup/output
2026-07-06 22:40:18,107 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager
at /0.0.0.0:8032
2026-07-06 22:40:19,560 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not per
formed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-06 22:40:19,575 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/ha
doop-yarn/staging/hadoop/.staging/job_1783352407947_0001
2026-07-06 22:40:19,771 INFO input.FileInputFormat: Total input files to process : 1
2026-07-06 22:40:19,820 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-06 22:40:19,920 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783352407947_0001
2026-07-06 22:40:19,920 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-06 22:40:20,056 INFO conf.Configuration: resource-types.xml not found
2026-07-06 22:40:20,057 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-06 22:40:20,404 INFO impl.YarnClientImpl: Submitted application application_1783352407947_0001
2026-07-06 22:40:20,431 INFO mapreduce.Job: The url to track the job: http://VIETBUINAM.localdomain:80
88/proxy/application_1783352407947_0001/
2026-07-06 22:40:20,432 INFO mapreduce.Job: Running job: job_1783352407947_0001
2026-07-06 22:40:25,514 INFO mapreduce.Job: Job job_1783352407947_0001 running in uber mode : false
2026-07-06 22:40:25,515 INFO mapreduce.Job:  map 0% reduce 0%
2026-07-06 22:40:30,569 INFO mapreduce.Job:  map 100% reduce 0%
2026-07-06 22:40:32,617 INFO mapreduce.Job:  map 100% reduce 100%
2026-07-06 22:40:32,626 INFO mapreduce.Job: Job job_1783352407947_0001 completed successfully
2026-07-06 22:40:32,678 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=70
                FILE: Number of bytes written=635715
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=4863109
                HDFS: Number of bytes written=64
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=2301
                Total time spent by all reduces in occupied slots (ms)=-217
                Total time spent by all map tasks (ms)=2301
                Total time spent by all reduce tasks (ms)=-217
                Total vcore-milliseconds taken by all map tasks=2301
                Total vcore-milliseconds taken by all reduce tasks=-217
                Total megabyte-milliseconds taken by all map tasks=2356224
                Total megabyte-milliseconds taken by all reduce tasks=-222208
```

Figure 33: Successful completion of the MapReduce job (map 100%, reduce 100%).

Result

The MapReduce job completed successfully. The HDFS output was exported to the local machine as the TSV-formatted file `results.txt`.

```
hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000
f       15870
i       15220
t       25223
h       18662
c       38934
m       25194
u       23790
s       50571
hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000 > results.txt
```

Figure 34: Exporting the output results.